



南京大學

NANJING UNIVERSITY

科研月报

学生姓名: _____

学生学号: _____

2026 年 6 月

1. 项目目标

本项目的目标是搭建一个低深度量子神经网络模型，用于图像角点与关键点检测中的 patch-level 二分类任务。模型不直接处理整张图像，而是对图像中的每一个候选中心点 $p = (x_0, y_0)$ ，输入其局部特征向量 x_p ，输出该点为角点或关键点的概率：

$$x_p \mapsto p_{\text{corner}}(p) \in [0, 1].$$

本计划书只关注 QNN 模型搭建、训练、测量输出和消融实验设计。图像读取、梯度计算、Harris 特征提取等可以由前处理模块完成，QNN 模块只需要接收已经构造好的特征矩阵和标签。

2. 任务定义

训练数据记为

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

其中

$$x_i \in \mathbb{R}^d$$

是第 i 个候选点对应的局部图像特征， $y_i \in \{0, 1\}$ 是标签：

$$y_i = \begin{cases} 1, & \text{中心点是 corner/keypoint,} \\ 0, & \text{中心点不是 corner/keypoint.} \end{cases}$$

第一版推荐使用五维特征：

$$x_p = [I_x(p), I_y(p), \lambda_1(p), \lambda_2(p), R(p)].$$

其中：

- $I_x(p), I_y(p)$ 是中心点附近的水平和竖直方向图像梯度；
- $\lambda_1(p), \lambda_2(p)$ 是以 p 为中心的局部窗口 structure tensor 的两个特征值；
- $R(p)$ 是 Harris response：

$$R(p) = \det M(p) - k \operatorname{tr}(M(p))^2.$$

structure tensor 定义为

$$M(p) = \sum_{(u,v) \in W} w(u,v) \begin{pmatrix} I_x(x_0 + u, y_0 + v)^2 & I_x(x_0 + u, y_0 + v)I_y(x_0 + u, y_0 + v) \\ I_x(x_0 + u, y_0 + v)I_y(x_0 + u, y_0 + v) & I_y(x_0 + u, y_0 + v)^2 \end{pmatrix}.$$

这里 W 是局部窗口， (u, v) 是相对于中心点的偏移， $w(u, v)$ 是窗口权重函数，例如 Gaussian 权重。

3. QNN 输入接口

Codex 需要实现的 QNN 模块应当接收如下输入：

- $\mathbf{X}_{\text{train}}$: shape 为 (N_{train}, d) 的特征矩阵；
- $\mathbf{y}_{\text{train}}$: shape 为 $(N_{\text{train}},)$ 的二分类标签；
- $\mathbf{X}_{\text{val}}$: shape 为 (N_{val}, d) 的验证集特征矩阵；
- $\mathbf{y}_{\text{val}}$: shape 为 $(N_{\text{val}},)$ 的验证集标签。

第一版设定

$$d = 5.$$

后续可扩展到八维特征：

$$x_p = [I_x, I_y, I_x^2, I_y^2, I_x I_y, \lambda_1, \lambda_2, R].$$

4. 特征归一化与角度映射

由于 angle encoding 需要将经典特征作为量子旋转门的角度，不能直接使用原始数值。对每一维特征 x_j ，先使用训练集统计量进行标准化：

$$z_j = \frac{x_j - \mu_j}{\sigma_j + \epsilon},$$

其中 μ_j 和 σ_j 分别是训练集中第 j 个特征的均值和标准差， ϵ 是防止除零的小常数，例如

$$\epsilon = 10^{-8}.$$

为了避免异常值造成过大的旋转角，对 z_j 做截断：

$$z_j \leftarrow \text{clip}(z_j, -3, 3).$$

然后映射到角度区间：

$$\phi_j = \frac{\pi}{3} z_j.$$

因此最终输入量子线路的是

$$\phi_p = [\phi_1, \phi_2, \dots, \phi_d],$$

且

$$\phi_j \in [-\pi, \pi].$$

代码中需要实现一个 `FeatureNormalizer` 类，具有以下功能：

- 在训练集上计算并保存 μ_j, σ_j ；

- 对训练集、验证集、测试集使用同一组 μ_j, σ_j ;
- 输出角度特征 ϕ ;
- 支持保存和加载归一化参数。

5. 推荐 QNN 结构

第一版推荐使用五量子比特、三层 data re-uploading VQC。每个特征对应一个 qubit:

$$q_0 \leftrightarrow I_x, \quad q_1 \leftrightarrow I_y, \quad q_2 \leftrightarrow \lambda_1, \quad q_3 \leftrightarrow \lambda_2, \quad q_4 \leftrightarrow R.$$

总线路为

$$|\psi_p\rangle = U(\phi_p, \theta)|0\rangle^{\otimes n},$$

其中

$$n = 5.$$

总变分线路写成

$$U(\phi_p, \theta) = \prod_{\ell=1}^L U_\ell(\phi_p, \theta_\ell),$$

其中 L 是 re-uploading 层数，第一版推荐

$$L = 3.$$

每一层定义为

$$U_\ell(\phi_p, \theta_\ell) = U_{\text{ent}} U_{\text{var}}(\theta_\ell) U_{\text{enc}}(\phi_p).$$

注意实际作用顺序为:

$$U_{\text{enc}} \rightarrow U_{\text{var}} \rightarrow U_{\text{ent}}.$$

6. Angle Encoding Layer

编码层定义为

$$U_{\text{enc}}(\phi_p) = \prod_{j=0}^{n-1} R_y^{(j)}(\phi_j) R_z^{(j)}(\phi_j).$$

也就是说，对第 j 个 qubit 执行:

$$R_y(\phi_j) R_z(\phi_j).$$

其中 R_y 将输入特征写入振幅分布， R_z 将输入特征写入相对相位。 R_z 并不是单独使用的，而是与后续可训练旋转和纠缠门配合，使相位信息转化为最终测量概率中的可见差异。

如果代码实现中想先做最简单版本，可以先支持:

$$U_{\text{enc}}^{(1)}(\phi_p) = \prod_{j=0}^{n-1} R_y^{(j)}(\phi_j),$$

再支持增强版本：

$$U_{\text{enc}}^{(2)}(\phi_p) = \prod_{j=0}^{n-1} R_y^{(j)}(\phi_j) R_z^{(j)}(\phi_j).$$

这两个版本应当作为消融实验选项。

7. Parameterized Single-Qubit Rotations

每一层的可训练单比特旋转层定义为

$$U_{\text{var}}(\theta_\ell) = \prod_{j=0}^{n-1} R_z^{(j)}(\theta_{\ell_j}^{(1)}) R_y^{(j)}(\theta_{\ell_j}^{(2)}) R_z^{(j)}(\theta_{\ell_j}^{(3)}).$$

每个 qubit 每层有三个可训练参数：

$$\theta_{\ell_j}^{(1)}, \theta_{\ell_j}^{(2)}, \theta_{\ell_j}^{(3)}.$$

对于 $n = 5, L = 3$ ，量子线路参数数量为

$$3 \times n \times L = 45.$$

这一参数量较小，适合 NISQ 约束下的小型变分分类器。

8. Ring Entanglement Layer

纠缠层采用 ring CNOT 结构：

$$U_{\text{ent}} = \prod_{j=0}^{n-1} \text{CNOT}_{j, (j+1) \bmod n}.$$

对于 $n = 5$ ，即：

$$\text{CNOT}_{0,1}, \quad \text{CNOT}_{1,2}, \quad \text{CNOT}_{2,3}, \quad \text{CNOT}_{3,4}, \quad \text{CNOT}_{4,0}.$$

如果实际硬件拓扑不支持 $q_4 \rightarrow q_0$ ，则实现 hardware-friendly 版本：

$$U_{\text{ent}}^{\text{linear}} = \prod_{j=0}^{n-2} \text{CNOT}_{j, j+1}.$$

代码中需要支持三种 entanglement 选项：

- **none**: 无纠缠；
- **linear**: 线性纠缠；
- **ring**: 环形纠缠。

9. Data Re-uploading 的实现方式

Data re-uploading 的核心是：同一个输入特征 ϕ_p 不只在最开始编码一次，而是在每一层都重新通过 $U_{\text{enc}}(\phi_p)$ 作用到当前量子态上。

因此，三层线路为

$$|\psi_p\rangle = U_3(\phi_p, \theta_3)U_2(\phi_p, \theta_2)U_1(\phi_p, \theta_1)|0\rangle^{\otimes n}.$$

展开后为

$$|\psi_p\rangle = [U_{\text{ent}}U_{\text{var}}(\theta_3)U_{\text{enc}}(\phi_p)][U_{\text{ent}}U_{\text{var}}(\theta_2)U_{\text{enc}}(\phi_p)][U_{\text{ent}}U_{\text{var}}(\theta_1)U_{\text{enc}}(\phi_p)]|0\rangle^{\otimes n}.$$

这里的 re-uploading 不是重置量子态，也不是重新制备输入态，而是在已经经过上一层训练变换与纠缠的当前量子态上，再次施加由同一个经典输入 ϕ_p 控制的编码门。

代码中应当通过循环实现：

```
for layer in range(L):  
  
    apply encoding gates depending on x  
  
    apply trainable single-qubit rotations  
  
    apply entanglement layer
```

其中输入数据 ϕ_p 不更新，可训练参数 θ 更新。

10. Final Measurement 与 Readout

推荐测量所有 qubit 的 Pauli-Z 期望值：

$$z_j(p) = \langle \psi_p | Z_j | \psi_p \rangle, \quad j = 0, \dots, n-1.$$

然后使用一个经典线性 readout：

$$s_p = b + \sum_{j=0}^{n-1} w_j z_j(p),$$

并通过 sigmoid 得到角点概率：

$$p_{\text{corner}}(p) = \sigma(s_p) = \frac{1}{1 + e^{-s_p}}.$$

其中 w_j, b 是经典可训练参数。

因此模型最终输出为

$$\hat{y}_p = p_{\text{corner}}(p).$$

代码中需要支持两种 readout：

- `single`: 只测量 $\langle Z_0 \rangle$;
- `all`: 测量所有 $\langle Z_j \rangle$ 并使用线性 readout。

第一版推荐使用 `all`。

11. 损失函数

使用 binary cross entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)].$$

由于角点样本通常少于非角点样本，需要支持 weighted BCE:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [\alpha y_i \log \hat{y}_i + (1 - \alpha)(1 - y_i) \log(1 - \hat{y}_i)].$$

也可以使用 PyTorch 中的 `BCEWithLogitsLoss(pos_weight=...)`, 此时模型输出 logits:

$$s_p = b + \sum_{j=0}^{n-1} w_j z_j(p),$$

不在模型内部手动做 sigmoid。

推荐代码实现采用 logits 版本，数值更稳定。

12. 训练流程

训练参数包括:

- 量子线路参数 θ ;
- 经典 readout 参数 w_j, b 。

输入特征 ϕ_p 不参与训练。

训练流程:

1. 从数据集中读取 batch:

$$\{(x_i, y_i)\}_{i=1}^B.$$

2. 使用训练集统计量将 x_i 标准化并映射为角度 ϕ_i 。

3. 对每个样本运行 QNN，得到 $z_j(i) = \langle Z_j \rangle$ 。

4. 使用经典 readout 得到 logit:

$$s_i = b + \sum_j w_j z_j(i).$$

5. 计算 BCE loss。
6. 反向传播计算梯度。
7. 使用 Adam 更新 θ, w, b 。

推荐超参数：

项目	推荐值
qubit 数 n	5
输入维度 d	5
re-uploading 层数 L	2 或 3
encoding	$R_y R_z$
variational layer	$R_z R_y R_z$
entanglement	ring CNOT
readout	all-qubit Z expectation + linear readout
loss	BCEWithLogitsLoss
optimizer	Adam
learning rate	10^{-3} 到 10^{-2}
batch size	32, 64 或 128
epochs	50 到 200
shots	模拟器先用 exact expectation；shot 模拟用 1024 或 2048

13. 代码模块设计建议

请 Codex 按如下模块实现。

13.1 数据归一化模块

文件名建议：

`preprocessing.py`

需要实现：

- `FeatureNormalizer.fit(X_train)`
- `FeatureNormalizer.transform(X)`
- `FeatureNormalizer.fit_transform(X_train)`
- `FeatureNormalizer.save(path)`
- `FeatureNormalizer.load(path)`

输出为角度特征 $\phi \in [-\pi, \pi]^d$ 。

13.2 QNN 线路模块

文件名建议：

`qnn_circuit.py`

需要实现一个类：

`DataReuploadingQNN`

初始化参数包括：

- `n_qubits`
- `n_layers`
- `encoding_type`: ry 或 ryrz
- `entanglement`: none, linear, ring
- `readout`: single, all
- `shots`: None 表示 exact expectation

核心 forward 过程：

1. 输入 batch 角度特征 Φ , shape 为 (B, d) ;
2. 对每个样本构建或执行量子线路;
3. 返回 logits, shape 为 $(B,)$ 。

如果使用 PennyLane, 推荐使用 `qml.qnode` 和 `qml.expval(qml.PauliZ(j))`。如果使用 Torch 接口, 需要保证参数可自动求导。

13.3 训练模块

文件名建议：

`train_qnn.py`

需要实现：

- 训练循环;
- 验证循环;
- loss 记录;

- accuracy, precision, recall, F1, ROC-AUC 或 PR-AUC;
- best model 保存;
- 随机种子设置;
- GPU/CPU 自动选择。

13.4 实验配置模块

文件名建议:

`config.py`

需要支持配置:

- 输入特征版本;
- qubit 数;
- re-uploading 层数;
- encoding 类型;
- entanglement 类型;
- readout 类型;
- learning rate;
- batch size;
- epochs;
- shots。

14. 必须完成的消融实验

为了证明 QNN 结构是否真正有效, 需要做以下消融。

14.1 输入特征消融

比较以下输入:

$$\begin{aligned}x^{(1)} &= [I_x, I_y], \\x^{(2)} &= [I_x, I_y, \lambda_1, \lambda_2], \\x^{(3)} &= [\lambda_1, \lambda_2, R], \\x^{(4)} &= [I_x, I_y, \lambda_1, \lambda_2, R].\end{aligned}$$

目的: 检查模型是否只是依赖 Harris response R , 还是确实利用了梯度与窗口结构信息。

14.2 编码方式消融

比较：

$$U_{\text{enc}}^{(1)} = \prod_j R_y(\phi_j),$$
$$U_{\text{enc}}^{(2)} = \prod_j R_y(\phi_j) R_z(\phi_j).$$

目的：验证加入 R_z 相位编码是否提升表达能力。

14.3 纠缠结构消融

比较：

`none, linear, ring.`

目的：检查不同局部特征之间的量子耦合是否有助于分类。

14.4 Data Re-uploading 层数消融

比较：

$$L = 1, \quad L = 2, \quad L = 3.$$

目的：检查反复输入数据是否增强模型表达能力。

14.5 经典模型对比

必须比较：

- Harris threshold baseline;
- classical logistic regression;
- classical MLP with same features;
- QNN without entanglement;
- QNN with entanglement;
- QNN with data re-uploading。

若 QNN 不能超过使用相同输入特征的 MLP，则不能声称 QNN 有性能优势，只能作为量子线路结构的基准实验或负结果分析。

15. 评价指标

第一阶段推荐使用 patch-level 分类指标：

- accuracy;
- precision;
- recall;
- F1 score;
- ROC-AUC;
- PR-AUC。

由于角点样本通常较少，PR-AUC 和 F1 比 accuracy 更重要。

第二阶段可加入 keypoint-level 指标：

- repeatability;
- localization error;
- matching score;
- number of detected keypoints;
- RANSAC inlier ratio。

但本计划书第一版代码只要求实现 patch-level QNN 分类。

16. 第一版代码实现目标

Codex 第一版应当完成：

1. 输入已经提取好的特征矩阵 X 和标签 y ;
2. 对特征进行标准化、截断和角度映射;
3. 实现 5-qubit data re-uploading QNN;
4. 支持 R_y 和 $R_y R_z$ 两种 encoding;
5. 支持 none、linear、ring 三种 entanglement;
6. 支持 $L = 1, 2, 3$ 的 re-uploading 层数;
7. 支持 all-qubit measurement readout;

8. 使用 BCEWithLogitsLoss 训练;
9. 输出训练日志和验证集指标;
10. 保存 best model;
11. 提供一个 run_ablation.py 脚本自动运行消融实验。

17. 推荐默认实验配置

默认配置如下:

$$\begin{aligned}
 x_p &= [I_x, I_y, \lambda_1, \lambda_2, R], \\
 n &= 5, \\
 L &= 3, \\
 U_{\text{enc}} &= \prod_{j=0}^4 R_y^{(j)}(\phi_j) R_z^{(j)}(\phi_j), \\
 U_{\text{var}}(\theta_\ell) &= \prod_{j=0}^4 R_z^{(j)}(\theta_{\ell_j}^{(1)}) R_y^{(j)}(\theta_{\ell_j}^{(2)}) R_z^{(j)}(\theta_{\ell_j}^{(3)}), \\
 U_{\text{ent}} &= \prod_{j=0}^4 \text{CNOT}_{j, (j+1) \bmod 5}, \\
 s_p &= b + \sum_{j=0}^4 w_j \langle Z_j \rangle, \\
 p_{\text{corner}}(p) &= \sigma(s_p).
 \end{aligned}$$

训练时使用 logits s_p 进入 BCEWithLogitsLoss, 不要在模型内部提前 sigmoid。

18. 可扩展方向

第一版跑通后, 可以扩展:

1. 八维输入:

$$[I_x, I_y, I_x^2, I_y^2, I_x I_y, \lambda_1, \lambda_2, R].$$

2. 加入 Hessian 特征:

$$[I_{xx}, I_{xy}, I_{yy}, \det H, \text{tr } H].$$

3. 用 4-qubit feature-split re-uploading 减少 qubit 数。
4. 加入 shot noise 模拟。
5. 加入 depolarizing noise 或 amplitude damping noise。
6. 与 small CNN 和 MLP 做更严格对比。

19. 总结

本项目的 QNN 部分建议实现为一个低深度 data re-uploading variational quantum classifier。它以局部角点特征

$$[I_x, I_y, \lambda_1, \lambda_2, R]$$

为输入，通过 angle encoding 将连续特征写入量子旋转门，再通过可训练单量子比特旋转和 ring entanglement 学习特征之间的非线性耦合。Data re-uploading 使同一个输入在多层线路中反复作为门参数出现，从而在不显著增加 qubit 数的情况下提升表达能力。最终测量所有 qubit 的 Pauli-Z 期望值，并通过经典线性 readout 输出角点概率。

第一版代码应重点保证结构清楚、可调参数明确、消融实验方便，而不是追求复杂线路。““

参考文献